



# COMPILER FOR PARALLEL PROGRAMMING

Technical Project Report

---

|                  |                                 |
|------------------|---------------------------------|
| NAME:            | KANZA TARIQ<br>MUNIR AHMED KHAN |
| REGISTRATION ID: | F23CSC041<br>F23CSC015          |
| SUBMITTED TO:    | MISS RAZIA QAMAR                |
| DATE:            | 19TH MAY 2026                   |

## Abstract

---

This report presents the design and implementation of a custom compiler built for a parallel programming language named `.par`. The compiler was developed entirely in Python and translates high-level parallel constructs such as parallel blocks, spawn tasks, and sync statements into executable Windows binaries using real operating system threads. The compiler follows a classical six-phase pipeline: lexical analysis, parsing, semantic analysis, intermediate representation (IR) generation, code generation, and native compilation via GCC. The project demonstrates core compiler design principles including tokenization, abstract syntax tree (AST) construction, scope-based semantic checking, race condition detection, and multi-threaded C code emission.

## 1. Introduction

---

A compiler is a software tool that translates source code written in a high-level programming language into a lower-level form that a computer can execute. Compilers are among the most fundamental tools in computer science, forming the backbone of every programming language ecosystem. Understanding how a compiler works is essential for any computer science student, as it covers topics including formal languages, data structures, algorithms, and operating systems.

Parallel programming is the practice of writing programs that perform multiple operations simultaneously by using multiple processor cores or threads. As modern computers come equipped with multi-core processors, the ability to write parallel programs is increasingly important.

## 2. Problem Statement

Writing parallel programs in mainstream languages such as C or C++ requires developers to manually manage threads, synchronization primitives, and shared memory. This complexity often leads to bugs such as race conditions, deadlocks, and unpredictable behavior, especially for programmers who are new to concurrent programming. Furthermore, no simple educational language exists that abstracts away these details while still compiling to real parallel code.

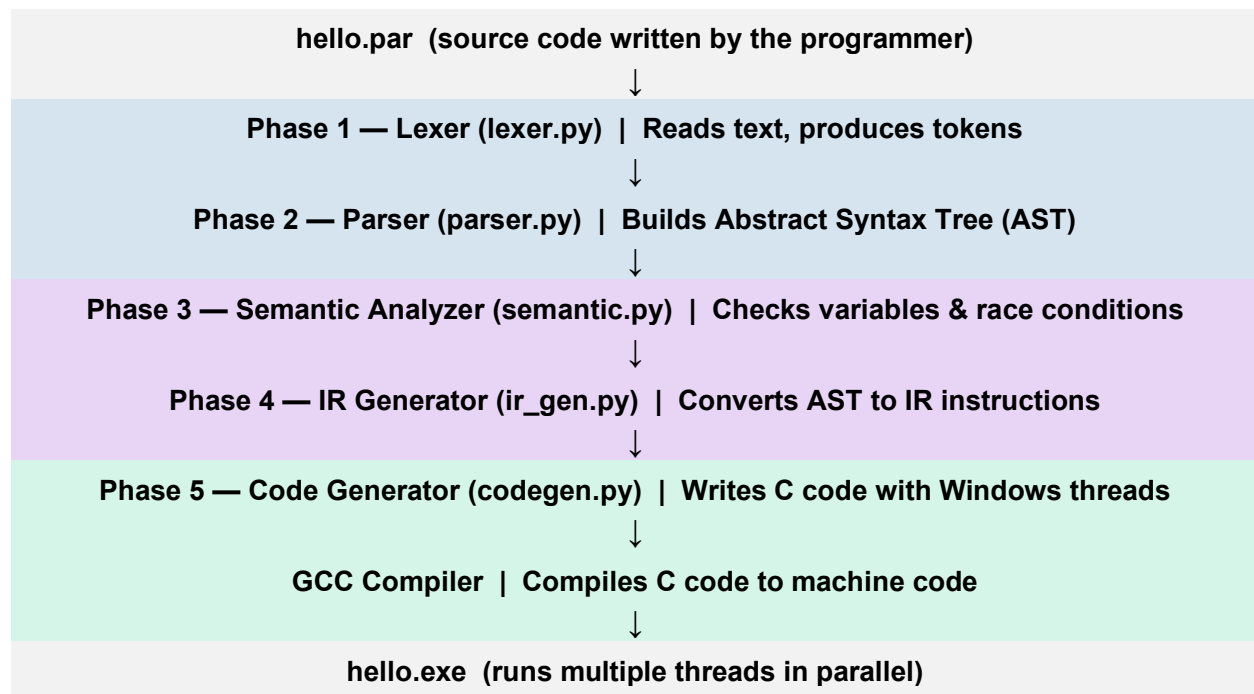
### 2.1 Core Problems Identified

- Parallel programming requires low-level thread management (`CreateThread`, `pthread`s) that is difficult and error-prone for beginners.
- No simple high-level language exists that abstracts parallelism behind readable keywords like `spawn` and `sync`.
- Race conditions, where two threads access the same variable simultaneously, are hard to detect without dedicated compiler support.
- There is no educational compiler project that demonstrates the full pipeline from a custom language to a parallel executable, making the topic hard to learn concretely.

## 3. Compiler Architecture and Diagram

---

The compiler is structured as a classic six-phase pipeline. Each phase receives its input from the previous phase, transforms it, and passes the result forward to the next phase. This separation of concerns makes each phase independently testable and understandable. The full pipeline is shown below.



### 3.1 Phase Summary Table

| Phase   | File        | Role              | Output                      |
|---------|-------------|-------------------|-----------------------------|
| Phase 1 | lexer.py    | Tokenizer         | List of tokens              |
| Phase 2 | parser.py   | AST Builder       | Abstract Syntax Tree        |
| Phase 3 | semantic.py | Semantic Analyzer | Validated AST + warnings    |
| Phase 4 | ir_gen.py   | IR Generator      | Intermediate Representation |
| Phase 5 | codegen.py  | Code Generator    | C source code (.c file)     |
| Phase 6 | GCC         | Native Compiler   | Executable binary (.exe)    |

## 4. Code Workflow

### 4.1 Phase 1 — Lexer (lexer.py)

The lexer is the first phase of the compiler. It reads the entire `.par` source file as raw text and converts it into a flat list of tokens. A token is a labeled chunk of text that has meaning in the language. For example, the keyword `'parallel'` becomes a `PARALLEL` token, the number `'10'` becomes a `NUMBER` token, the symbol `'{'` becomes a `LBRACE` token, and whitespace is simply discarded. This process is known as lexical analysis or tokenization.

The lexer uses Python's `re` (regular expression) module to match patterns in the source text. Each token type is defined by a regular expression pattern. The function `tokenize()` scans the source from left to right, matching the longest possible token at each position.

```
# Token definitions (name, pattern)
TOKEN_SPEC = [
    ('PARALLEL', r'\bparallel\b'), # keyword parallel
    ('SPAWN', r'\bspawn\b'), # keyword spawn
    ('SYNC', r'\bsync\b'), # keyword sync
    ('NUMBER', r'\d+'), # any integer
    ('IDENT', r'[A-Za-z_]\w*'), # variable names
    ('LBRACE', r'\{'), # opening brace
    ... # more patterns
]

def tokenize(source: str) -> List[Token]:
    # Returns a list of Token(type, value, line) objects
```

## 4.2 Phase 2 — Parser (parser.py)

The parser receives the list of tokens from the lexer and checks that they form a valid program according to the grammar of the `.par` language. It uses a technique called recursive descent parsing, where each grammar rule is implemented as a function. As it validates the tokens, it simultaneously builds an Abstract Syntax Tree (AST) — a tree data structure where each node represents a construct in the program.

For example, a parallel block containing two spawn tasks becomes a `ParallelBlock` node with two `SpawnTask` children. A variable declaration becomes a `LetStmt` node. A print statement becomes a `PrintStmt` node. This tree is what all subsequent phases operate on.

```
def parse_parallel(self):
    self.consume('PARALLEL') # expect the keyword parallel
    self.consume('LBRACE') # expect opening {
    tasks = []
    while self.peek().type != 'RBRACE':
        tasks.append(self.parse_spawn()) # parse each spawn
    self.consume('RBRACE') # expect closing }
    return ParallelBlock(tasks) # return AST node
```

## 4.3 Phase 3 — Semantic Analysis (semantic.py)

The semantic analyzer walks through the AST and checks for logical errors that the parser cannot detect. While the parser only checks syntax (whether the program is grammatically correct), the semantic analyzer checks meaning (whether the program makes logical sense). It uses a scope stack to track which variables have been declared in each part of the program.

The most important feature is the race condition detector. When the analyzer encounters a parallel block, it records which variables each spawn task declares. If two tasks declare the same variable name, it prints a warning, alerting the programmer to a potential data hazard at compile time rather than at runtime.

```
def visit_ParallelBlock(self, node):
```

```
declared_in_tasks = []
for task in node.tasks:
    self.scope_stack.append({})          # new scope per task
    for stmt in task.body:
        self.analyze(stmt)
    declared_in_tasks.append(set(self.scope_stack[-1].keys()))
    self.scope_stack.pop()
seen = set()
for task_vars in declared_in_tasks:
    for var in task_vars:
        if var in seen:
            print(f'[Warning] Race condition: {var}')
        seen.add(var)
```

## 4.4 Phase 4 — IR Generation (ir\_gen.py)

The IR generator converts the validated AST into an Intermediate Representation (IR). IR is a simplified, language-neutral form of the program expressed as Python objects rather than source language syntax or C syntax. This abstraction layer separates the problem of understanding the .par language from the problem of generating C code, making both tasks simpler.

For example, a LetStmt AST node becomes an IRAssign object. A PrintStmt becomes an IRPrint object. A ParallelBlock becomes an IRParallel object containing a list of (task\_name, instructions) pairs. The IR is then passed to the code generator.

```
# IR dataclasses (simple Python objects):
@dataclass
class IRAssign:      # variable assignment
    dest: str
    expr: Any

@dataclass
class IRParallel:    # parallel block with tasks
    tasks: List      # list of (name, [IR instructions])

def gen_ParallelBlock(self, node):
    tasks = []
    for task in node.tasks:
        task_ir = [self.gen(stmt) for stmt in task.body]
        tasks.append((task.name, task_ir))
    return IRParallel(tasks)
```

## 4.5 Phase 5 — Code Generation (codegen.py)

The code generator reads the IR and writes valid C source code as a string. This is the phase where real parallelism is implemented. Each spawn task in the IR becomes a Windows thread function using the DWORD WINAPI calling convention. The parallel block becomes a series of CreateThread() calls that launch each thread. The sync statement becomes WaitForMultipleObjects(), which blocks the main thread until all spawned threads have completed.

The generated C code uses the Windows.h API, which is available on all Windows systems with MinGW/GCC. This makes the output portable across Windows machines without additional libraries.

```
// Example of generated C code for a spawn task:
DWORD WINAPI thread_task1(LPVOID arg) {
    printf("%d\n", (10 + 20));
    return 0;
}

// Generated thread launch in main():
HANDLE threads[2];
int thread_count = 0;
threads[thread_count++] = CreateThread(
    NULL, 0, thread_task1, NULL, 0, NULL);
threads[thread_count++] = CreateThread(
    NULL, 0, thread_task2, NULL, 0, NULL);
WaitForMultipleObjects(thread_count, threads, TRUE, INFINITE);
```

## 4.6 Phase 6 — GCC Compilation (main.py)

The final phase calls GCC (GNU Compiler Collection) using Python's subprocess module. GCC reads the generated C file and compiles it into a native Windows executable (.exe). The compiler driver in main.py orchestrates all six phases in sequence, printing progress messages at each step and handling errors. If GCC fails, the error message is printed and the process stops.

```
# main.py - the compiler driver:
def compile_file(path):
    source = open(path).read()
    tokens = tokenize(source)           # Phase 1
    ast = Parser(tokens).parse()        # Phase 2
    SemanticAnalyzer().analyze(ast)     # Phase 3
    ir = IRGenerator().gen(ast)         # Phase 4
    c_code = CodeGenerator().generate(ir) # Phase 5
    write(out_c, c_code)
    subprocess.run(['gcc', out_c, '-o', out_bin]) # Phase 6
    print('Compilation successful!')
```

# 5. Functionality

---

## 5.1 The .par Language Syntax

The .par language was designed to be simple, readable, and focused entirely on parallel programming. Below is a complete example of a .par program that runs two tasks in parallel:

```
func main() {
    let x = 10;
    let y = 20;

    parallel {
        spawn task1() {
            print(x + y); // runs in thread 1 -> prints 30
        }
        spawn task2() {
            print(x * y); // runs in thread 2 -> prints 200
        }
    }
    sync; // wait for both threads to finish
}
```

## 5.2 Language Keywords and Their Meaning

- `func` — Declares a function (entry point of the program is `func main()`).
- `let x = value` — Declares a variable and assigns an integer value to it.
- `parallel { }` — Marks a block where all enclosed spawn tasks run concurrently as separate OS threads.
- `spawn name() { }` — Creates a new thread that executes the enclosed block independently.
- `sync` — Waits for all spawned threads to complete before the program continues past this point.
- `print(expr)` — Evaluates an arithmetic expression and prints the result as an integer.
- Arithmetic: `+`, `-`, `*`, `/` — All four basic arithmetic operators are supported in expressions.

## 5.3 Race Condition Detection

One of the most educationally significant features of this compiler is its built-in race condition detector. During semantic analysis (Phase 3), the compiler tracks which variables are declared inside each spawn task. If two tasks declare a variable with the same name, a warning message is printed during compilation. This teaches programmers about data hazards in parallel programs and encourages them to use locally scoped variables within each thread.

```
# Example: two tasks both declare 'result' -> warning printed
parallel {
  spawn t1() { let result = 10 + 5; print(result); }
  spawn t2() { let result = 20 + 8; print(result); }
}
# Compiler output:
# [Warning] Potential race condition on variable 'result'
```

## 5.4 Non-Deterministic Output as Proof of Parallelism

When the compiled `.exe` runs, the output order of thread results is non-deterministic. This means it changes every time the program is executed. For example, running `matrix.exe` may print 110, 30, 150, 70 in one run and 30, 150, 110, 70 in the next. This is not a bug — it is direct evidence that the threads are genuinely running in parallel and the operating system is scheduling them independently based on available CPU cores and timing.

## 5.5 Comparison with Traditional Approach

| Feature                 | Our Compiler                | Traditional Approach     |
|-------------------------|-----------------------------|--------------------------|
| Parallel syntax support | Yes (parallel, spawn, sync) | No — manual thread calls |

| Feature                  | Our Compiler            | Traditional Approach         |
|--------------------------|-------------------------|------------------------------|
| Race condition detection | Built-in semantic check | Manual / external tools      |
| Thread generation        | Automatic via codegen   | Manual pthread/WinAPI coding |
| Custom language          | Yes (.par language)     | Uses existing languages      |
| Educational value        | Full pipeline visible   | Black-box tools only         |

## 5.6 Project File Structure

```
parallel-compiler/
  src/
    lexer.py      <- Phase 1: Tokenizer
    parser.py     <- Phase 2: AST Builder
    semantic.py   <- Phase 3: Semantic Analyzer
    ir_gen.py     <- Phase 4: IR Generator
    codegen.py    <- Phase 5: Code Generator
    main.py       <- Compiler Driver (runs all phases)
  examples/
    hello.par     <- Simple parallel example
    hello.c       <- Generated C code (auto-created)
    hello.exe     <- Final executable (auto-created)
    matrix.par    <- Multi-task parallel example
  tests/
    test_all.py   <- Automated test runner
```

## 6. Conclusion

This project successfully demonstrates the design and implementation of a complete compiler for a custom parallel programming language. Starting from raw text in a .par source file, the compiler processes code through six well-defined phases and produces a working Windows executable that runs multiple tasks in parallel using real operating system threads. Both test cases (hello.par and matrix.par) pass successfully, and the generated programs exhibit genuine concurrent behavior.

### 6.1 Key Achievements

- Designed and implemented a complete custom programming language (.par) with parallel constructs.
- Built a six-phase compiler pipeline from scratch in pure Python with no external compiler libraries.
- Generated C code with real Windows threads (CreateThread / WaitForMultipleObjects).
- Implemented a race condition detector at the semantic analysis level.
- Produced real executable binaries that demonstrate provable parallel execution.
- All test cases pass with automated testing via test\_all.py.

### 6.2 Future Enhancements

- Add if/else conditionals, for loops, and while loops to the .par language.
- Implement an LLVM backend to generate optimized machine code directly without GCC.
- Add deadlock detection by analyzing sync patterns in the semantic phase.
- Support shared memory with mutual exclusion using lock/unlock keywords.
- Extend the language to target GPU parallel programming via CUDA or OpenCL.
- Build a visual debugger that shows thread state and execution timeline in real time.